

Frontend Standards

A folder you copy into a React/Next.js project. No install, no plugin registry, no build step.

v1.3 — atomic design · BFM-grounded UX · accessibility at the atom · Playwright page review · performance budgets · security · PWA

THE ONE IDEA THAT MATTERS

The package is split by **who owns each file**. That split is what makes updating safe.

TIER 1 — GENERIC

Reusable across every project. **Overwritten on update**. Don't edit these.

- 17 skills
- 3 agents, 4 commands
- hooks, boundary checker, page reviewer

TIER 2 — YOURS

True only in this project. **Never touched by an update**.

- project-conventions skill
- its failure-modes log
- CLAUDE.md , CI

WHAT THIS PACKAGE INSISTS ON

Five things, beyond the usual:

RULE	WHY
Atomic design is the base of implementation — atom → molecule → organism → template → page	Every level renders in isolation, so all four states get <i>looked at</i> . Extraction after the fact produces components shaped like one screen's accident.
Accessibility is built into the atom , not audited later	A molecule cannot fix an inaccessible atom, only wrap it. Retrofitting means auditing every consumer.
UX is grounded in a cited BFM pattern , queried before the checklist	Interaction design invented on the spot looks finished, ships, and fails quietly where a user gives up.
You drive the page before calling it done	Reading the code tells you what should render. Driving it tells you what does.

Page speed is a budget you measure —
LCP, CLS, JS and total weight

The reviewer exits non-zero when one is exceeded. A
budget nobody measures is a wish in a document.

atom molecule organism template page
Button → SearchField → PostCard → FeedLayout → /posts
Input FormRow NavBar AuthLayout /sign-in

one-way, always. only the page fetches data.

WHAT'S IN THE BOX

```
frontend-standards/
├── .claude/
│   ├── skills/           17 generic + your project-conventions
│   ├── agents/          code-reviewer, ui-reviewer, debugger
│   ├── commands/        /review /ship /debug /adr-new
│   ├── hooks/           format-on-edit, commit guard (Node)
│   └── settings.json
├── scripts/
│   ├── check-boundaries.mjs  the mechanical half
│   └── review-page.mjs       drive the page with Playwright
├── .github/workflows/ci.yml
├── CLAUDE.md               short router
├── install.ps1             Windows
└── install.sh              macOS / Linux
```

FILE	EDIT PER PROJECT?
.claude/skills/project-conventions/	Yes — the important one
CLAUDE.md	Yes — your layout
.github/workflows/ci.yml	Yes — package manager, script names
Everything else	No

WHERE THE FILES GO

The **contents** go into your project root. The `frontend-standards/` folder itself never enters your project — it stays where you unzipped it, as the source you re-run updates from.

```
your-app/
├─ .claude/
├─ scripts/check-boundaries.mjs
├─ .github/workflows/
├─ CLAUDE.md
├─ app/  components/  lib/      <- your code, untouched
└─ package.json
```

Collisions in an existing project

Nothing is ever overwritten. Where you already have a file, the package version lands beside it as `<name>.new` , and every one is listed at the end of the run.

YOU ALREADY HAVE	WHAT HAPPENS
<code>scripts/</code> , <code>.claude/agents,commands,hooks</code>	Merged. Your own files survive.
<code>.github/workflows/deploy.yml</code>	Merged per file — you still receive <code>ci.yml</code>
<code>.github/workflows/ci.yml</code> , <code>CLAUDE.md</code>	Yours kept; ours staged as <code>.new</code>
<code>.claude/settings.json</code>	Yours kept; staged as <code>.new</code> — merge the <code>hooks</code> block

An unmerged `.new` file enforces nothing

The installer lists every one at the end so it can't be missed. Diff each pair, merge what you want, then delete the `.new` .

SETUP

1 Unzip it somewhere permanent

Not `Downloads` — you return here every time you start or update a project.

PowerShell

```
Expand-Archive frontend-standards-dropin.zip -DestinationPath C:\dev\  
cd C:\dev\frontend-standards
```

bash

```
unzip frontend-standards-dropin.zip -d ~/dev/  
cd ~/dev/frontend-standards && chmod +x install.sh
```

No shell dependency

Hooks are Node, not shell scripts — no bash requirement, no `chmod` on the hooks, no PowerShell/bash divergence. Node is already a given in a React project.

2 Copy it into your project

```
.\install.ps1 C:\dev\your-app          # PowerShell  
./install.sh /path/to/your-app         # bash
```

Same behaviour either way

`install.ps1` writes the Windows variant of `settings.json` (`%CLAUDE_PROJECT_DIR%`); the bash one writes the POSIX variant (`$CLAUDE_PROJECT_DIR`). Both invoke `node .`

3 Run both checkers — see where you stand

```
node scripts/check-boundaries.mjs  
node scripts/review-page.mjs /          # needs the app running
```

Violations are fine on day one. Fix them, or relax the config and tighten later. The one rule worth keeping strict immediately is the client-import rule — it is the one that pays for itself.

4 Fill in tier 2 — the highest-value step

`.claude/skills/project-conventions/SKILL.md` ships as a template:

- **Settled decisions** — with the reason and what would reopen them
- **Product rules** — the invariants every screen inherits
- **Domain vocabulary** — words that mean something specific here
- **The contract** — how the client is generated, the envelope, the error codes
- **Design source** — what's ported, where provenance lives
- `references/failure-modes.md` — what has actually gone wrong here

Why the failure log earns its place

Generic standards encode what *usually* goes wrong. The failure log encodes what has gone wrong *in this codebase* — a far better predictor, and the only part you can't download. It ships seeded with a dozen rows that recur across frontends; keep the ones that can happen here.

5 Edit CLAUDE.md and CI

CLAUDE.md — the routing table works as-is; change the layout section to your actual tree. Keep it short: it loads on every turn.

ci.yml — match your package manager and script names. It assumes pnpm and `gen:api`.

6 Turn on the commit guard — last

The guard runs the boundary checker before every commit, so a leaked client import is caught when it's one file rather than thirty. Enable it once the rest is green, by merging the `hooks` block into `.claude/settings.json`.

THE BOUNDARY CHECKER

This is the mechanical half of the standards — the part that doesn't rely on anyone remembering. Three rules, zero dependencies:

RULE	WHY IT MATTERS
No component imports the generated API client	One contract change should touch one directory, not thirty components

Imports never point upward — atom → molecule → organism → template → page

A level that reaches up can no longer render in isolation

No raw hex in components

A literal is a copy no design update will ever reach

Zero-config — it recognises both `atoms/molecules/organisms/templates` and the older `ui/patterns/states/layout` naming, and skips directories that don't exist, so it's safe in a partially-migrated repo. Configure in `package.json` when yours differs:

```
{
  "checkBoundaries": {
    "root": "src",
    "clientModules": ["lib/api/generated"],
    "vendorSeams": { "@vendor/sdk": ["lib/vendor"] },
    "allowRawHex": ["components/brand", "components/icons"]
  }
}
```

It fails if it scans zero files

A check that finds nothing to scan must fail, or it reports green while enforcing nothing. Exit 0 = clean, 1 = violations, 2 = misconfigured.

THE PAGE REVIEWER

The other half of the mechanical enforcement — and the one that catches what no linter can.

```
node scripts/review-page.mjs /posts
node scripts/review-page.mjs /posts --states    # force the failure path
```

Per width (390px and 1440px) it captures:

- A full-page **screenshot**
- Console errors and failed/4xx requests, *with the URL*
- Horizontal overflow — the classic responsive break

- A **keyboard focus-order trace**, flagging any stop without a visible ring
- An **axe accessibility scan**
- **Core Web Vitals and transfer weight**, checked against budgets

```
{ "reviewPage": { "budgets": { "lcpMs": 2500, "cls": 0.1, "jsKb": 250, "totalKb": 1000 } } }
```

Retrofitting? Set the budgets to your current numbers

Then ratchet down. A budget set at the ideal on day one fails every build for something nobody in that PR caused, and gets disabled within a week.

What it does not measure

INP

needs a real interaction, so it is absent rather than reported as a meaningless zero. Weight comes from content-length, so a response without that header is not counted. These are

lab numbers on your machine's connection

— field data beats them.

Playwright and `@axe-core/playwright` are optional — the script degrades and says which part it skipped rather than failing. Set `REVIEW_BROWSER_PATH` if your image ships its own Chromium.

Look at the screenshots

A green report with an unreadable page is still a broken page. No scan sees "the heading overlaps the card". And the three non-success states are only covered if you force them — which is exactly where the design gave you nothing.

THE WORKFLOW THE SKILLS ENFORCE

Discuss → Confidence Check → Checklist → Definition of Done → [CHECK-IN GATE] → Implement → Doc

Three constraints do most of the work:

- **Confidence is scored out of 10.** Below 8, resolve the blockers and re-score before proceeding.
- **The check-in gate is unconditional** — approval before any implementation, at any score, even 10/10. The plan response contains no tool calls. A response that proposes a plan and

starts executing it hasn't been approved; it's been announced.

- **BFM is queried at step 1–2**, before the checklist. A pattern found after implementation is a rewrite.
- **Deep checks report findings only.** Reviews and debugs never edit until the fix and its impact are approved.

SKILLS

You don't invoke these. Claude loads one when the work matches its description.

SKILL	FIRES WHEN
build-workflow	Starting any work; decomposing a large build; blocked by a missing API or design
ux-patterns	Before designing any screen — query the BFM MCP, cite the pattern
page-review	Before saying a screen looks right — drive it with Playwright
performance	Adding a dependency, image, font, or animation; a page feels slow
security-checker	Raw HTML, tokens, URLs from users, a new dependency, or anything personal
pwa-offline	Service worker, install, caching, or a dropped connection
project-conventions	Tier 2 — how <i>this</i> project does it
project-structure	Where a file goes; import direction; promoting to a shared tier
component-standards	Atomic levels; writing or styling a component; adding a variant
ui-states	Any view that fetches — the four states
data-layer	Hooks, mutations, invalidation, undo buffers, polling
forms-validation	Any form; multi-step flows
accessibility	Anything interactive or laid out — build-time, not review-time

frontend-testing	Tests, Storybook, e2e, flaky or hanging suites
analytics	Tracking, events, a metric that looks wrong
design-sync	Porting from a design source; recording a divergence
adr · release-notes	Decisions; cutting a release

Agents and commands

NAME	DOES
code-reviewer	Structure — boundaries, data layer, tests
ui-reviewer	What the user gets — four states, copy, a11y, dead controls, unbacked claims
/review	Runs the gate, spawns both reviewers, applies the triggered checklists
/ship	Verifies every Definition-of-Done criterion before you call it done

UPDATING A PROJECT LATER

```
cd C:\dev\frontend-standards
.\install.ps1 C:\dev\your-app      # or ./install.sh
```

Tier 1 is refreshed. Tier 2 — your project-conventions , failure log, CLAUDE.md , and CI — is left alone.

`--force / -Force` **overwrites your project knowledge**
It replaces project-conventions and the failure log with blank templates. Commit before running it.

TROUBLESHOOTING

SYMPTOM	CAUSE
---------	-------

Skills never load	<code>.claude/</code> isn't at the repo root, or Claude opened in a parent directory
Hooks do nothing	<code>node</code> not on PATH; check the commands in <code>.claude/settings.json</code>
Checker exits 2	Scanned zero files — wrong directory, or <code>root</code> needs setting in <code>package.json</code>
Checker flags a legitimate icon colour	Add that directory to <code>allowRawHex</code>
Checker misses a violation	Your alias isn't <code>@/</code> or <code>~/</code> — it resolves those two plus relative paths
Guard blocks a legitimate <code>console.log</code>	It only checks app source. If it fired, the file is under <code>app/</code> , <code>components/</code> , or <code>lib/</code>
<code>review-page</code> says playwright missing	<code>pnpm add -D playwright @axe-core/playwright</code> , then <code>pnpm exec playwright install chromium</code>
<code>review-page</code> can't launch a browser	Set <code>REVIEW_BROWSER_PATH</code> to an existing Chromium binary
Budgets fail on an existing project	Expected. Set them to your current numbers in <code>package.json</code> , then ratchet down
Weight reads lower than the network tab	Only responses sending <code>content-length</code> are counted
Project conventions vanished after an update	<code>--force</code> was used; recover from git

Three rules, if you keep nothing else: **build bottom-up from accessible atoms, no component imports the generated client**, and **anything true only in this project lives in tier 2**.